

1. High-Level Architecture

The goal is to move from siloed systems to a centralized data warehouse (or a unified operational database) with ETL/ELT processes to ingest and transform data.

Suggested architecture:

1. Source Systems

- Student Information System (SIS)
- Faculty Management System
- Course Management System
- Library Management System

2. Integration Layer

- ETL tools (e.g., Apache NiFi, Talend, or cloud services like AWS Glue)
- Change Data Capture (CDC) for real-time or near-real-time sync
- Data validation and cleansing steps

3. Centralized Database

- Option A: Operational Data Store (ODS) for real-time queries and transactional needs
- Option B: Data Warehouse for analytics and reporting
- Could use a cloud-based solution (Amazon RDS, Azure SQL, Google Cloud SQL) or a modern data warehouse (Snowflake, BigQuery)

4. Consumption Layer

- Reporting & BI tools (Tableau, Power BI)
- APIs for departmental access
- Dashboards for real-time metrics

2. Unified Data Model

A star schema or 3NF model depending on whether the focus is analytics (star) or operational consistency (3NF). Below is a simplified logical model.

Entity	Attributes (examples)
Student	student_id (PK), first_name, last_name, date_of_birth, email, enrollment_status, department_id (FK)
Faculty	faculty_id (PK), first_name, last_name, email, department_id (FK), hire_date, title
Department	department_id (PK), name, budget_code, dean_id (FK to Faculty)
Course	course_id (PK), course_code, title, credits, department_id (FK), description
CourseOffering	offering_id (PK), course_id (FK), faculty_id (FK), academic_year, semester, capacity, enrolled_count, location
Enrollment	enrollment_id (PK), student_id (FK), offering_id (FK), enrollment_date, grade, status
LibraryLoan	loan_id (PK), student_id (FK), faculty_id (FK), resource_id, checkout_date, return_date, status
LibraryResource	resource_id (PK), title, type (book, journal, etc.), isbn, location

Relationships

- Student → Enrollment → CourseOffering → Course
- Faculty → CourseOffering (teaches)
- Faculty → Department (belongs to)
- Department → Course (offers)
- Student/Faculty → LibraryLoan → LibraryResource

3. Addressing Key Issues

a. Data Redundancy & Inconsistency

- Source: Different systems store student name, email, etc., with variations.
- Solution:
 - Use master data management (MDM) principles: designate SIS as the source of truth for student data.
 - Build ETL transformations to standardize formats, deduplicate, and apply business rules.
 - Implement data quality checks (e.g., no duplicate student_id across sources).

b. Single Point of Access

- Solution:
 - Create unified views (materialized or virtual) combining student, faculty, and course data.
 - Expose these via a data access layer (REST APIs) so all departments query the same source.

c. Manual Intervention

- Solution:
 - Automate ETL pipelines with scheduling (e.g., daily incremental loads, or CDC for real-time).
 - Use error handling and alerting to notify admins only when exceptions occur.

d. Real-Time Reporting

- Solution:
 - Implement Change Data Capture from source systems to update the central DB with minimal latency.
 - Use a columnar database or data warehouse optimized for read queries.
 - Pre-aggregate key metrics like:
 - Enrollment counts by course/offering
 - Faculty teaching load
 - Course availability (remaining seats)

e. Security Concerns

- Solution:
 - Centralize role-based access control (RBAC) at the database and API level.
 - Encrypt sensitive data (PII) at rest and in transit.
 - Mask or anonymize data for non-production use.
 - Maintain audit logs of who accessed what data.